

HW1: Linear and Neural Sentiment Classification

Nicholas McCracken
mccracken.161@osu.edu

1 Exploration 1: Bigram and Better Feature Extractors

Two alternative feature extractors are implemented as an extension of the base Unigram Feature Extractor. This includes a bigram feature extractor and a custom "better" feature extractor. Both inherit from the FeatureExtractor parent class and make use of an Indexer.

The bigram feature extractor extracts over adjacent word pairs. While this significantly increases expressive power, it led to strong overfitting in practice. The training accuracy became nearly perfect, but development accuracy drops relative to unigrams. This suggests that the bigrams capture highly specific lexical patterns, which do not generalize very well under the limits of the movie review training data. With some additional regularization, or perhaps by combining unigram and bigram feature extraction, a more effective result could be achieved.

In contrast, I designed the better feature extractor to focus on generalization in hopes to address these problems. A small hand-crafted set of stop words are removed from the vocabulary, and unigram counts are clipped to a maximum of three occurrences per sentence. These methods reduce the influence of highly frequent words while still preserving the sentiment cues that are vital to correct predictions. Comparing this to the standard Unigram Feature Extractor, better features slightly improved the development accuracy and F1.

A nice benefit of using these specific feature modifications is the unigram feature space is reused, so training time is and the weight vector are not affected in any way. For further improvements, iterative experimentation like discarding rare words, and using bigrams or trigrams could be implemented. However, this would increase training time.

2 Exploration 2: CNN Architecture

A simple one-layer 1D convolutional neural network (CNN) is implemented as an alternative architecture to the required Deep Averaging Network (DAN). I added number of filter and kernel size parameters, which were set to 100 and 3 respectively for the experiments. This convolution layer is followed by a ReLU activation and global max pooling. Padding is explicitly added to the convolution layer so that sentences shorter than the kernel size can be processed without runtime errors.

Unlike DAN, which represents sentences by averaging embeddings and therefore discards the word order entirely, CNN preserves the local word order information through its convolutional filters. This advantage allows the model to capture short patterns. For example negation phrases like "not good". This relationship is invisible to the DAN model.

The CNN is trained using the same optimization loop as DAN, and thus they can easily be compared. The CNN achieves the highest development accuracy and F1 score among all the tested models. Even a shallow convolutional structure provides a great advantage over unordered sentence representations.

3 Discussion

Both explorations show that increasing depth or parameter count of a model is not always necessary for better results. Bigram feature extraction increases representational power and capturing of relationships, but generalizes poorly. The "better" feature extractor improves performance through filtering alone. Similarly, the CNN outperforms DAN by providing a greater understanding of sentence structure.

Model	Dev Accuracy	Dev F1
LR (UNIGRAM)	0.7523	0.7787
LR (BIGRAM)	0.7339	0.7537
LR (BETTER)	0.7683	0.7878
DAN	0.7821	0.7865
CNN	0.8016	0.8080

Table 1: Development set performance across models.